Other Ways to Combine Data in SQL

DATA SCIENCE BOOTCAMP

NASHVILLE SOFTWARE SCHOOL

Lesson Objectives

1. Understand the key concepts of Set Theory
2. Be able to execute queries using:
    - UNION
    - UNION ALL
    - INTERSECT
    - EXCEPT

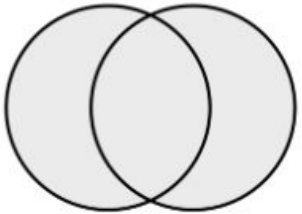
Set Theory Clauses

These clauses can be used to apply set theory to SQL queries. The four clauses available are:

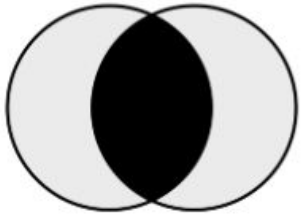
- UNION
- UNION ALL
- INTERSECT
- EXCEPT

Unlike joins, these clauses are used to combine data **without the use of keys**. When using these clauses, the tables involved **must have the same columns** because instead of pairing rows from one table up with matching rows in another to merge the tables together, UNION and UNION ALL stack one table on top of another while INTERSECT and EXCEPT result in a table that resembles the original tables, but with only rows that match certain criteria still included.

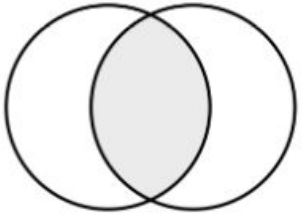
Set Theory Clauses



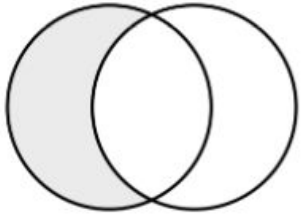
UNION



UNION ALL



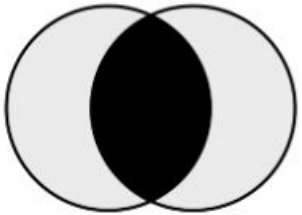
INTERSECT



EXCEPT

UNION

UNION takes all rows from both tables and removes duplicates, so if the same row occurs in both tables, it is only represented once.



UNION can be helpful when the tables you want to combine don't have different information on the same items, but rather have similar types of items that are not the same OR you don't want them to be treated as the same.

One thing to remember when combining tables this way, **all included columns in the tables must match in content and listed order**, since a UNION will simply place one on top of another.

UNION - Example

Using Olympics data that has all of the summer games in one table and all of the winter games in another, let's say we wanted a list of all the games from both seasons.

```
(SELECT event
FROM summer_games)
UNION
(SELECT event
FROM winter_games);
```

event character varying (255)
Men's 5,000 meters
Women's Triple Jump
Men's Marathon
Gymnastics Men's Horizontal Bar
Women's 400 metres
Cross Country Skiing Men's 50 kilometers

Each game is listed **exactly once**, even if the game shows up multiple times in one of the original tables.

UNION - Example

Something that might be helpful in our new table is another column that indicates which season the game is from. The original tables do not have this column.

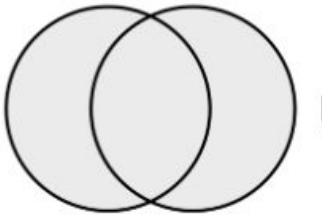
```
(SELECT event,
        'Summer' AS season
FROM summer_games)
UNION
(SELECT event,
        'Winter' AS season
FROM winter_games);
```

Simply list the text you want to be present in your new column in quotes and alias it as the new column name in both SELECT statements.

event character varying (255)	**season** text
Men's Pole Vault	Summer
Men's 4 X 100 metres Relay	Summer
Women's 4 X 100 metres Relay	Summer
Gymnastics Men's Rings	Summer
Cross Country Skiing Men's 50 kilometres Freestyle	Winter
Swimming Women's 200 metres Freestyle	Summer

UNION ALL

UNION ALL is the same thing as UNION except **all instances of a duplicate rows will be included**.



Always keep in mind that this duplication can occur whether the duplicates exist within one table or are the result of combining the two tables.

If you incorporate DISTINCT in each of the SELECT statements, you would then **only** get duplication that is caused by combining the tables.

UNION ALL - Example

If we execute the same query as before except replace the UNION with UNION ALL, we can see duplication occurring.

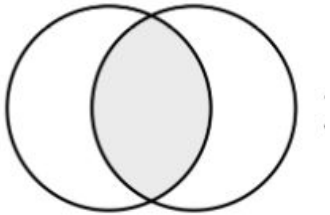
```
(SELECT event,
        'Summer' AS season
FROM summer_games)
UNION ALL
(SELECT event,
        'Winter' AS season
FROM winter_games);
```

event character varying (255)	**season** text
Gymnastics Men's Individual All-Around	Summer
Gymnastics Men's Floor Exercise	Summer
Gymnastics Men's Parallel Bars	Summer
Gymnastics Men's Horizontal Bar	Summer
Gymnastics Men's Rings	Summer
Gymnastics Men's Pommelled Horse	Summer
Men's 5,000 metres	Summer
Men's 400 metres	Summer
Men's 5,000 metres	Summer

We can see that there is duplication occurring now, although the duplicates in this table snippet are from the same table and not caused by combining the table.

INTERSECT

Instead of stacking one table on the other like we saw in UNION and UNION ALL, **INTERSECT** returns only rows that occur in both tables.



Like in the previous clauses, make sure to pay attention to the order and content of the columns being included in each SELECT statement. These should match to achieve accurate results.

INTERSECT - Example

This time, let's look at the athlete_id column of the summer_games and winter_games tables to see if there are any athletes that have competed in both.

```
(SELECT athlete_id
FROM summer_games)
INTERSECT
(SELECT athlete_id
FROM winter_games);
```

athlete_id integer
127594

The resulting table shows that only athlete in this data set who competed in both sets of games. This table only gives us an ID though and doesn't show anything about who this person is.

INTERSECT - Example

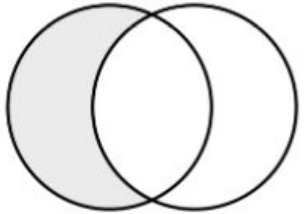
It is possible to use any set theory clause in conjunction with other clauses we would use to modify a table. Let's use JOIN to bring in the athletes table to get this athlete's name.

```
(SELECT athlete_id,
        name
FROM summer_games
        LEFT JOIN athletes
        ON summer_games.athlete_id = athletes.id)
INTERSECT
(SELECT athlete_id,
        name
FROM winter_games
        LEFT JOIN athletes
        ON winter_games.athlete_id = athletes.id);
```

athlete_id integer	**name** character varying (255)
127594	Eva Vrabcov-Nvltov

EXCEPT

EXCEPT is the opposite of INTERSECT. EXCEPT will include everything from the first table that doesn't exist in the second table.



Unlike INTERSECT, the **order you list your tables in is important** when using EXCEPT.

EXCEPT - Example

Let's use EXCEPT to see which countries are represented in the Summer Olympics, but not the Winter Olympics in this dataset.

```
(SELECT country_id
FROM summer_games)
EXCEPT
(SELECT country_id
FROM winter_games);
```

country_id integer
38
150
139
140

EXCEPT - Example

We can use JOIN like we did with earlier to gather more information about the countries represented in summer but not winter.

```
(SELECT country
FROM summer_games
        LEFT JOIN countries
        ON summer_games.country_id = countries.id)
EXCEPT
(SELECT country
FROM winter_games
        LEFT JOIN countries
        ON winter_games.country_id = countries.id);
```

country character varying (255)
MDV - Maldives
Lao - Laos
T.To - Trinidad and Tobago
SEN - Senegal

EXCEPT - Example

Now, let's only return countries in Oceania.

```
(SELECT country
FROM summer_games
        LEFT JOIN countries
        ON summer_games.country_id = countries.id
WHERE region = 'OCEANIA')
EXCEPT
(SELECT country
FROM winter_games
        LEFT JOIN countries
        ON winter_games.country_id = countries.id);
```

country character varying (255)
SAM - Samoa
FIJ - Fiji
PNG - Papua New Guinea
GUM - Guam

In this case, we only need to use a WHERE in the first table, since it doesn't matter if the second table is limited to Oceania.

EXCEPT - Example

Finally, let's put our table in alphabetical order.

```
(SELECT country
FROM summer_games
        LEFT JOIN countries
        ON summer_games.country_id = countries.id
WHERE region = 'OCEANIA')
EXCEPT
(SELECT country
FROM winter_games
        LEFT JOIN countries
        ON winter_games.country_id = countries.id)
ORDER BY country;
```

country character varying (255)
COK - Cook Islands
PNG - Papua New
A.Sa - American Samoa
F.SM. - Federated States of Micronesia
FIJ - Fiji
GUM - Guam

We get some weird results here. We won't really get into string methods for SQL, but for now, just notice that we likely have some leading spaces in some country names

A Note on Clause Placement

When adding additional clauses to your set clause statement, make sure to give plenty of thought to which SELECT statement the clause needs to act on. Some rules to consider are:

- WHERE and HAVING are table-specific. If you need both SELECT statements involved in the set clause to be filtered, you will need to have a WHERE twice. It is also possible to filter one and not the other, or both by different things.
- If you want your final output to be sorted, ORDER BY goes at the very end to sort the whole table at once.
- If aggregations are necessary, you will need to aggregate and GROUP BY in each table separately.

Compare & Contrast: Joins vs. Set Theory Clauses

Joins

- Use keys to match values in rows
- You can select any columns you want in any order in your SELECT statement
- Combines data horizontally; selecting all available columns would result in a wider table

Set Theory Clauses

- Does not use keys, no matching of rows
- Must have columns that align with each other (order matters) in each SELECT statement
- Combines data vertically; the selected columns must match up, final table will have the same columns

Additional Resources

- https://www.postgresql.org/docs/current/queries-union.html
- https://www.sqlshack.com/learn-sql-set-theory/
- https://arctype.com/blog/postgres-set-operators/
- https://app.datacamp.com/learn/courses/joining-data-in-sql Chapter 3
- https://app.datacamp.com/learn/courses/sql-for-joining-data Chapter 3